

任务目标

使用 Faker 和 MSW 完成最小可用的模拟接口环境，并实现一个简化版接口文档页，能够在线发送请求并查看响应。

完成后的效果

访问 `/api-docs` 后，可以看到接口列表。点击“发送请求”后，页面会显示接口状态码和 JSON 响应内容。健康检查接口能返回固定种子 2026，重置接口能恢复初始模拟数据。

你需要完成什么

1. 安装 `@faker-js/faker` 和 `msw`。
2. 使用固定种子生成稳定模拟数据。
3. 创建共享数据仓库。
4. 使用 MSW 拦截开发环境接口。
5. 使用 Axios 请求模拟接口。
6. 创建 `/api-docs` 接口文档页。

请求链路

本任务的核心链路是：

```
ApiDocsView 页面
-> src/api/mock.js
-> src/api/request.js
-> MSW handler
-> mockStore
-> JSON 响应
```

页面不要直接导入数据数组，必须通过 Axios 请求接口。

目录结构要求

任务完成后至少应有以下结构：

```
src/
├── api/
│   ├── mock.js
│   └── request.js
├── mock/
│   ├── browser.js
│   ├── database/
│   │   ├── seed.js
│   │   └── store.js
```

```
|   └─ handlers/
|       └─ index.js
└─ views/
    └─ api-docs/
        └─ ApiDocsView.vue
```

操作步骤

步骤 1：安装依赖并生成 worker

安装依赖：

```
npm install @faker-js/faker msw
```

确认项目中存在：

```
public/mockServiceWorker.js
```

步骤 2：创建固定种子数据

在 `src/mock/database/seed.js` 中完成：

1. 导入 `fakerZH_CN`。
2. 设置 `faker.seed(2026)`。
3. 设置固定参考日期 `2026-01-01`。
4. 生成 5 条用户数据。

用户数据建议包含：

- `id`
- `name`
- `phone`
- `role`
- `createdAt`

步骤 3：创建共享数据仓库

在 `src/mock/database/store.js` 中完成：

- `read()`：读取当前数据。
- `reset()`：重新生成初始数据。
- `localStorage` 持久化。

要求：刷新页面后数据仍能读取，点击重置后恢复固定种子数据。

步骤 4：创建 MSW handlers

在 `src/mock/handlers/index.js` 中完成两个必做接口：

方法	地址	响应
GET	<code>/api/health</code>	服务状态、数据版本、种子、用户数量
POST	<code>/api/mock/reset</code>	重置后的版本、种子、用户数量

统一响应格式：

```
{
  code: 0,
  message: '操作成功',
  data: {}
}
```

步骤 5：启动 MSW

在 `src/mock/browser.js` 中导出 worker。

在 `src/main.js` 中只在开发环境启动：

```
if (import.meta.env.DEV) {
  const { worker } = await import('./mock/browser')
  await worker.start({ onUnhandledRequest: 'bypass' })
}
```

步骤 6：封装 Axios 请求

在 `src/api/request.js` 中创建 Axios 实例：

- `baseUrl` 设置为 `/api`
- `timeout` 设置为 10000

在 `src/api/mock.js` 中封装：

- `getMockHealth()`
- `resetMockData()`

步骤 7：创建接口文档页

在 `src/views/api-docs/ApiDocsView.vue` 中完成：

1. 定义接口数组，包括方法、路径、说明、请求函数。
2. 渲染接口列表。
3. 每个接口提供“发送请求”按钮。
4. 点击后显示：

- HTTP 状态码
- 请求耗时
- JSON 响应内容

步骤 8：接入路由和菜单

在 `src/router/index.js` 中新增：

```
/api-docs
```

在 `src/router/menu.js` 中新增“接口文档”菜单。

接口清单

方法	地址	说明
GET	<code>/api/health</code>	获取模拟服务状态和数据版本
POST	<code>/api/mock/reset</code>	重置模拟数据

选做接口

方法	地址	说明
GET	<code>/api/mock/error</code>	模拟 500 异常
ALL	<code>/api/*</code>	模拟 404 异常

自测步骤

1. 启动项目。
2. 访问 `/api-docs`。
3. 点击 GET `/api/health` 的发送按钮。
4. 检查响应中是否有：
 - `code: 0`
 - `status: healthy`
 - `seed: 2026`
 - `users: 5`
5. 点击 POST `/api/mock/reset` 的发送按钮。
6. 检查是否返回“模拟数据已重置”。

验收清单

- ☐ `/api-docs` 页面可以打开。

- ☐ 页面至少展示健康检查和数据重置两个接口。
- ☐ 点击健康检查可以看到 200 响应。
- ☐ 响应中可以看到种子 2026 。
- ☐ 点击重置接口可以看到重置成功响应。
- ☐ 响应格式为 { code, message, data }。
- ☐ 页面没有直接导入模拟数据数组。
- ☐ 时间允许时补充 404 和 500 异常接口。
- ☐ 时间允许时为接口文档页增加“请求参数”区域。

易错提醒

常见错误	现象	解决方式
MSW 没有启动	请求返回 404 或走真实网络	检查 main.js 是否启动 worker
handlers 地址写错	接口不被拦截	地址必须和 Axios 请求路径对应
每次请求都重新 Faker	数据不稳定	Faker 只在初始化和重置时调用
错误响应无法显示	页面报错或空白	使用 error.response?.data 读取错误响应

任务内选做

完成基础验收后，如课堂时间允许，继续完成以下选做项：

1. 补充 404 和 500 异常接口。
2. 在接口文档页新增“请求参数”区域。
3. 思考客户管理模块需要哪些接口，并写出接口清单。